hexagone
École Supérieure d'Informatique

Git et GitHub

1GEST – Gestion de projet et outils collaboratifs

Chris CHEVALIER

hexagone
École Supérieure d'Informatique

Sommaire

#1 Pourquoi versionner ?

#2 Les notions de Git

#3 GitHub Desktop pas à pas

#4 Travailler en équipe sur GitHub

#5 Souveraineté numérique

#1 Pourquoi versionner ?

Git et GitHub

Vous connaissez déjà ce dossier

Dossier « Projet »

```
rapport.docx
rapport-v2.docx
rapport-v2-relu-par-Manon.docx
rapport-final.docx
rapport-final-v3.docx
rapport-final-v3-VRAIMENT-final.docx
```

- Quelle est la **bonne** version ?
- Qui a modifié quoi ? Pourquoi ?
- Comment travailler à trois **sans écraser** le travail des autres ?

> Le **versionnement** répond à ces questions : il enregistre chaque étape du projet avec son auteur et sa raison.

Git et GitHub : deux choses différentes

Git · le logiciel

- Installé **sur votre ordinateur**
- Enregistre l'historique du projet
- **Libre et gratuit**

GitHub · l'hébergeur

- Un **site web** qui garde une copie du projet en ligne
- Ajoute des outils d'équipe : tickets, relecture, tableaux
- Appartient à **Microsoft**

Logiciel libre : logiciel que chacun peut utiliser, étudier, modifier et redistribuer. D'autres hébergeurs existent : GitLab, Codeberg…

#2 Les notions de Git

Git et GitHub

Le commit : une photo de votre projet

- Un **commit** est une photo du projet, avec un message qui résume le changement
- Il garde son **auteur** et sa **date**
- On peut revenir à n'importe quelle photo

Message	Avis
« modifs »	✗
« update README.md »	✗
« Ajout des user stories du sprint 1 »	✓

> Un bon message dit **ce qui a changé**, sans avoir à ouvrir le commit.

Du dépôt local au dépôt distant

Dépôt local : le dossier du projet sur votre ordinateur. **Dépôt distant** : sa copie sur GitHub.

1 · Pull

- Récupérer le travail de l'équipe, **avant de commencer**

2 · Modifier

- Écrire, corriger, enregistrer

3 · Add

- Choisir les fichiers à mettre dans le commit

4 · Commit

- Valider l'étape avec un message clair

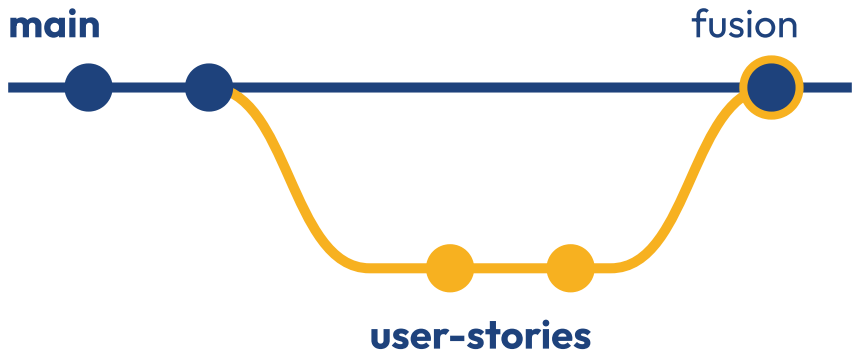
5 · Push

- Envoyer vos commits sur GitHub

Un commit reste **sur votre ordinateur** tant qu'il n'est pas poussé. Dépôt (*repository*) : dossier suivi par Git. *Pull* : tirer. *Add* : ajouter. *Push* : pousser.

Les branches

- Une **branche** est une ligne de travail à part : vous avancez sans gêner les autres
- `main` contient la version **validée** du projet
- Une branche par tâche, avec un nom clair : `user-stories`



> Quand la tâche est terminée, on **fusionne** la branche dans `main`. On parle de **merge**.

La pull request : faire relire avant de fusionner

- Une **pull request** (PR) demande de fusionner une branche dans `main`
- Un coéquipier **relit**, commente puis approuve
- On fusionne sur GitHub : **Merge pull request**

Sur GitLab, on parle de *merge request*.

Exemple

Manon ouvre la PR « Ajout des user stories du sprint 1 ».

Cosmo signale une story sans critère d'acceptation. Manon corrige, la PR se met à jour.

Cosmo approuve, Chris fusionne.

Les conflits

- Un **conflit** apparaît quand deux personnes modifient **la même ligne**
- Git ne devine pas : il vous demande de choisir
- Ce n'est **pas une erreur**, c'est une décision

README.md (en conflit)

```
<<<<<<< HEAD
# Appli de covoiturage étudiant
=======
# CoVoit'Campus
>>>>>>> titre-projet
```

> **Résoudre** : pas besoin d'effacer les marqueurs à la main. VS Code affiche des boutons au-dessus du conflit pour garder **votre version**, **l'autre** ou **les deux**. Il reste à faire un commit.

Markdown : la syntaxe en une diapo

Markdown : du texte simple et quelques symboles. Fichier en `.md`, affiché mis en forme par GitHub.

J'écris	J'obtiens
`# Titre`	le titre du document (un seul)
`## Section` · `### Sous-section`	des titres de section
`**important** · *nuance*`	**important** · *nuance*
`- élément`	une liste à puces
`1. étape`	une liste numérotée
`- [ ] à faire` · `- [x] fait`	des cases à cocher

J'écris	J'obtiens
`[Trello](https://trello.com)`	un lien cliquable
`![Logo](docs/logo.png)`	une image
`` `docs/need.md` ``	un nom de fichier ou une commande
`> Citation`	un paragraphe en retrait
`| A | B | puis |---|---|`	un tableau

Une ligne vide sépare deux paragraphes. Toute la syntaxe : écriture et mise en forme sur GitHub. L'onglet **Preview** de l'éditeur de GitHub montre le résultat avant d'enregistrer.

Le README : la page d'accueil du dépôt

GitHub l'affiche sous la liste des fichiers : c'est la **première chose** qu'on lit en ouvrant votre dépôt.

En une minute, il répond à quatre questions :

- **Quoi ?** Le projet en deux phrases
- **Pour qui et pourquoi ?** Le client et son besoin
- **Où en est-on ?** L'état et les liens utiles
- **Qui ?** L'équipe et le rôle de chacun

Les sections de votre README

1. Titre et présentation du projet
2. Le client et son besoin
3. L'équipe et les rôles (tableau)
4. Les liens : Trello, projet GitHub
5. Les documents de `docs/` (liens)
6. L'avancement (cases à cocher)
7. La déclaration d'usage de l'IA

> Un README se lit en diagonale : titres clairs, listes courtes, liens.

Un README complet

README.md

```
# Grand Prix Hexagone

Application web du tournoi Mario Kart de la rentrée :
inscriptions, grille des courses et classement en direct.

## Le client et son besoin

Le **bureau des étudiants** veut en finir avec le papier.
Détails : [docs/need.md](docs/need.md)

## L'équipe

| Membre | Rôle          |
|--------|---------------|
| Manon  | Product Owner |
| Cosmo  | Scrum Master  |
| Chris  | Developer     |

## Liens

- [Tableau Trello](https://trello.com/b/…)
- [Projet GitHub](https://github.com/…/projects/1)

## Avancement

- [x] Besoin reformulé et validé
- [x] Backlog estimé
- [ ] Sprint 1 terminé

## Usage de l'IA

Le Chat (Mistral AI) a relu `docs/need.md`.
```

Un seul `#`, une section `##` par question, des **liens** plutôt que des copier-coller.

#3 GitHub Desktop pas à pas

Git et GitHub

Installer et se connecter

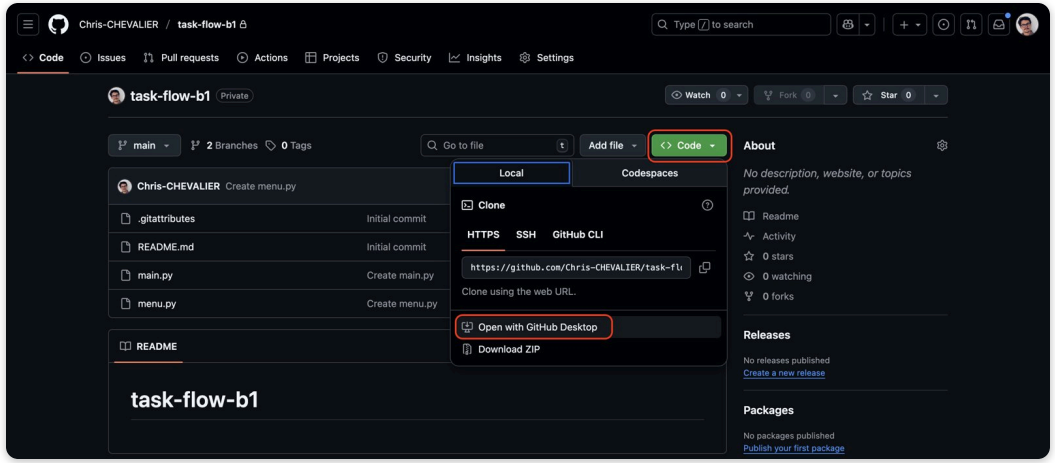
1. Créez un compte sur github.com et activez la **double authentification**
2. Installez **GitHub Desktop** et l'éditeur **VS Code**
3. Dans GitHub Desktop, cliquez sur **Sign in to GitHub.com** et autorisez l'application
4. Vérifiez votre nom et votre e-mail : ils signent vos commits

> **GitHub Desktop** utilise Git sans taper de commande : chaque bouton en lance une à votre place.

Double authentification : en plus du mot de passe, un code donné par une application du téléphone. Un mot de passe volé ne suffit plus.

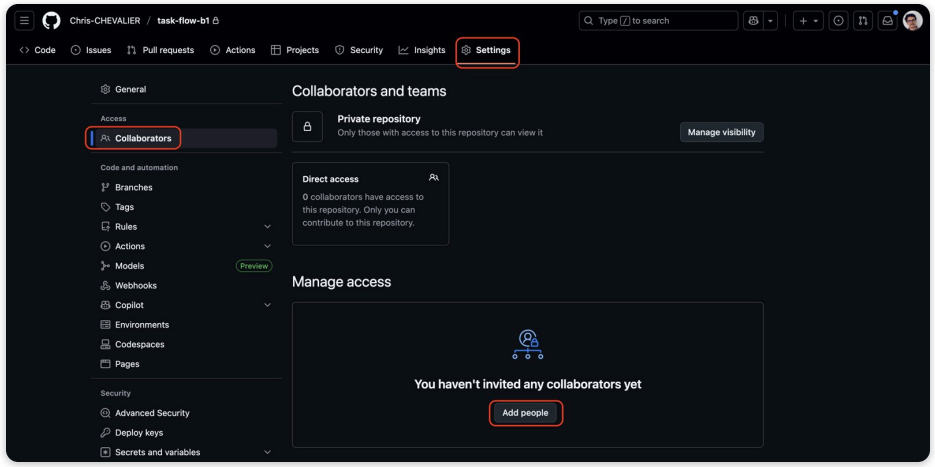
Récupérer le dépôt de l'équipe : cloner

1. Un membre crée le dépôt sur github.com : **New repository**, avec **Add a README file**
2. Il invite les autres membres
3. Chacun ouvre le dépôt sur github.com : **Code** → **Open with GitHub Desktop**
4. Choisissez un dossier puis cliquez sur **Clone**

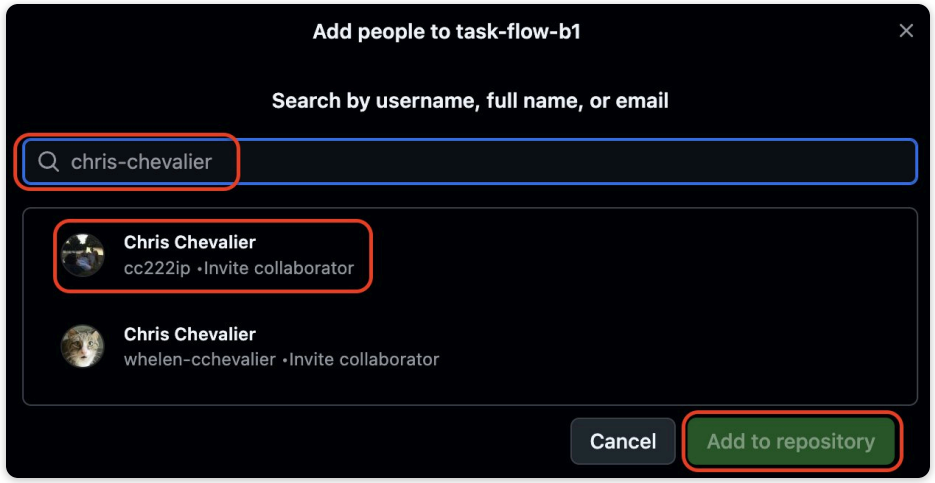


Cloner : télécharger une copie complète d'un dépôt, historique compris. Une seule fois par ordinateur. Autre chemin : **File** → **Clone repository** dans GitHub Desktop.

Inviter les membres de l'équipe



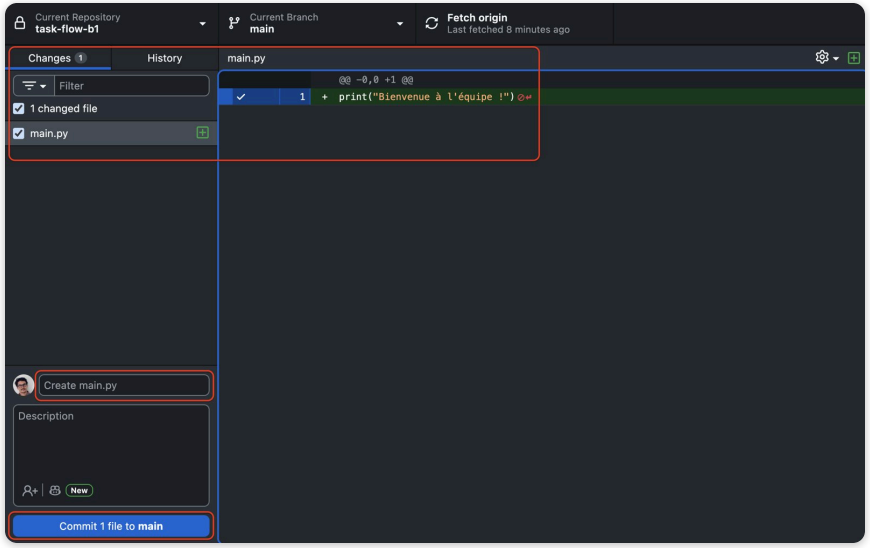
Sur github.com : **Settings** → **Collaborators** → **Add people**



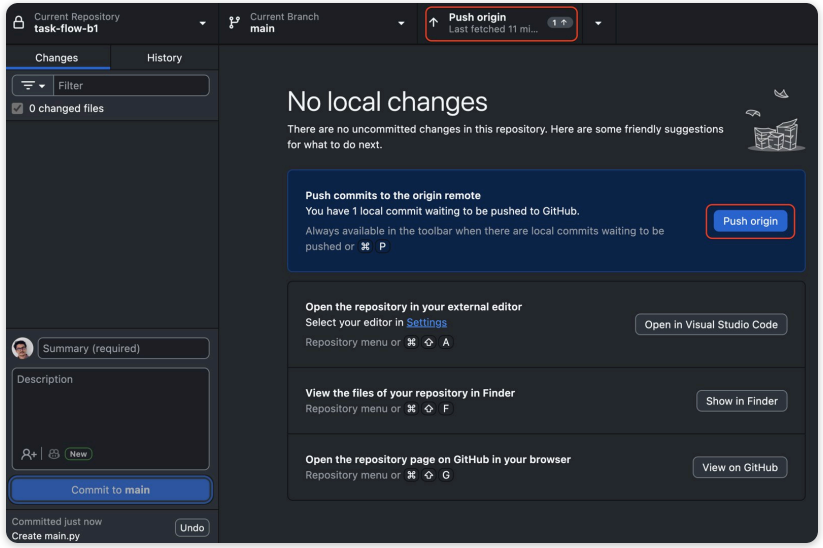
Saisissez le nom d'utilisateur, puis **Add to repository**. Le membre accepte l'invitation reçue par e-mail.

> Invitez aussi **Chris-CHEVALIER**. Je suis votre projet depuis le dépôt et c'est lui que je note.

Faire un commit puis le pousser



Changes : cochez les fichiers (Add), écrivez le message dans **Summary**, puis **Commit to main**.



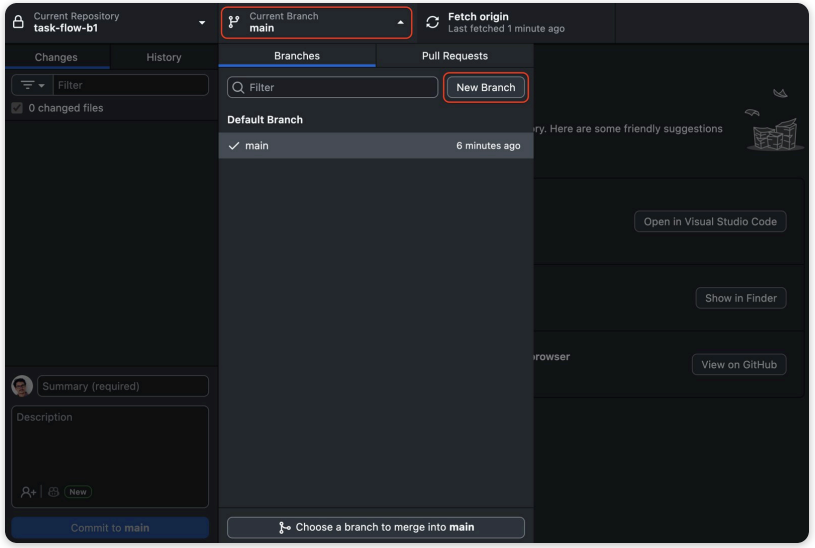
Puis **Push origin** envoie le commit sur GitHub.

> **Avant de commencer** : **Fetch origin**. S'il y a du nouveau, le bouton devient **Pull origin** : cliquez dessus.

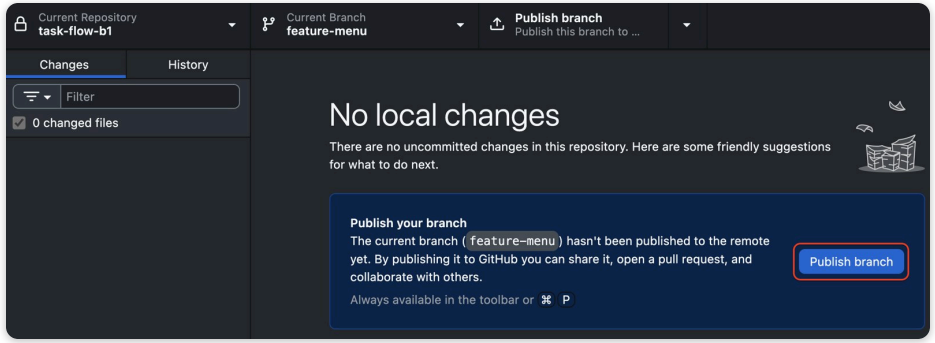
Démo Du dépôt au premier commit

1. Sur github.com : **New repository**, nom `1gest-demo-grand-prix`, **Private**, **Add a README file**, **Create repository**
2. **Code** → **Open with GitHub Desktop**, puis **Clone**
3. **Open in Visual Studio Code**, ouvrir `README.md` et y coller le README de la diapo « Un README complet »
4. Enregistrer, revenir dans GitHub Desktop : le fichier apparaît dans **Changes**
5. Summary : `Ajout de la présentation du projet`, puis **Commit to main**
6. **Push origin**, puis **View on GitHub** : le README s'affiche mis en forme

Créer une branche puis ouvrir une pull request



Current Branch → **New Branch**, un nom clair, **Create Branch**. Faites vos commits.



Publish branch, puis **Preview Pull Request** → **Create Pull Request** (vérifiez **base: main**).

> Sur github.com, un coéquipier relit (**Files changed** → **Review changes** → **Approve**). On fusionne avec **Merge pull request**, puis chacun revient sur `main` et fait un **Pull**.

Démo Une branche et une pull request

1. **Current Branch** → **New Branch** : `docs/need`
2. Dans VS Code, créer `docs/need.md` et coller : `# Besoin reformulé` puis `Objectif : inscrire 64 joueurs avant le 1er octobre.`
3. Commit `Ajout du besoin reformulé`, puis **Publish branch**
4. **Preview Pull Request** → **Create Pull Request**. Titre : `Ajout du besoin reformulé`. Description : `Closes #1`
5. Onglet **Files changed** : un commentaire sur une ligne, puis **Merge pull request**
6. Dans GitHub Desktop : revenir sur `main` et **Pull origin**

Résoudre un conflit

1. GitHub Desktop signale les fichiers en conflit
2. Pour chaque fichier, il propose de **garder une version entière** ou d'ouvrir le fichier dans l'éditeur : **Open in Visual Studio Code**
3. Dans VS Code, au-dessus du conflit : **Accept Current Change** (votre version), **Accept Incoming Change** (l'autre) ou **Accept Both Changes** (les deux)
4. Enregistrez, revenez dans GitHub Desktop, faites le commit puis poussez

> **Pour en avoir moins** : un pull avant de travailler, des branches courtes et prévenir l'équipe avant de modifier un fichier commun.

Ce que GitHub Desktop fait à votre place

Dans GitHub Desktop	Commande Git	Rôle
Clone repository	`git clone <adresse>`	copier un dépôt distant
Cocher les fichiers	`git add <fichier>`	préparer le commit
Commit to main	`git commit -m "message"`	enregistrer une étape
Push origin	`git push`	envoyer sur GitHub
Pull origin	`git pull`	récupérer le travail de l'équipe
New Branch	`git switch -c <nom>`	créer une branche et s'y placer
Merge into current branch	`git merge <branche>`	fusionner une branche

Pour information : ces commandes se tapent dans un terminal. Vous n'en avez pas besoin ici. Le livre Pro Git (en français)

#4 Travailler en équipe sur GitHub

Git et GitHub

Les issues : une tâche, un ticket

- Une **issue** est un ticket qui décrit une tâche ou un problème
- Un titre, une personne assignée (**Assignees**) et des étiquettes (**Labels**)
- `Closes #3` dans une PR ferme l'issue n° 3

#3 · Rédiger les user stories du sprint 1

documentation sprint 1

Assignée à **Manon** · 3 points

> La règle de l'équipe : **une issue, une branche, une pull request**, relue par un autre membre. Personne ne fusionne sa propre PR.

GitHub Projects : le tableau de bord du projet

Vue Board · le Kanban

- Les issues en colonnes : *Todo*, *In Progress*, *Done*
- On déplace la carte quand la tâche avance

Vue Roadmap · le Gantt

- Les mêmes issues sur une frise du temps
- Une date de début et une date de fin par issue

- Créer : **Projects** → **New project**, puis le relier au dépôt
- **Milestones** (jalons) : **Issues** → **Milestones**, une échéance qui regroupe des issues

Documentation GitHub Projects · la vue Roadmap

Démo Issues et GitHub Projects

1. **Issues** → **Milestones** → **New milestone** : `Sprint 1`, date de fin `11/12/2026`
2. **Issues** → **New issue** : `Rédiger le besoin reformulé`, assignée à moi, milestone `Sprint 1`
3. Deuxième issue `Saisie des scores (5)`, label `Must`, description avec `- [ ] Je choisis la course` et `- [ ] Le classement se met à jour`
4. **Projects** → **New project** → **Board** : `Grand Prix Hexagone`, puis relier le dépôt et ajouter les deux issues
5. Glisser une carte de *Todo* à *In Progress*
6. **New view** → **Roadmap** : ajouter les champs *Start date* et *Target date*, puis dater les issues

#5 Souveraineté numérique

Git et GitHub

Où vivent vos données ?

- GitHub appartient à **Microsoft**, une entreprise américaine
- Le **Cloud Act** permet à la justice américaine d'exiger ces données, même stockées en Europe

Souveraineté numérique : capacité à maîtriser ses outils et ses données sans dépendre d'un seul acteur. Le *Cloud Act* est une loi américaine de 2018.

Des alternatives ouvertes

- **GitLab** : libre et installable sur les serveurs de l'école
- **Forgejo** : forge libre, utilisée par **Codeberg**, association allemande

> **Git reste libre.** Votre dépôt et son historique se déplacent d'un hébergeur à l'autre sans rien perdre.

Les bons réflexes de sécurité

Votre compte

- **Double authentification** obligatoire
- Un mot de passe unique, rangé dans un gestionnaire de mots de passe
- N'invitez que les membres du projet

Votre dépôt

- **Jamais de mot de passe**, de clé d'accès ni de jeton dans un fichier versionné
- **Jamais de données personnelles** (adresses, téléphones, notes…)
- Dépôt **privé** par défaut

Supprimer un fichier ne suffit pas : **il reste dans l'historique**. Un secret poussé une fois est un secret à changer.

Ce que vous faites cet après-midi

Le dépôt

- Créé par un membre
- Équipe et Chris-CHEVALIER invités
- Cloné par chacun

Les issues

- Le backlog Trello devient des issues
- GitHub Projects : Board et Roadmap

Les livrables

- Une branche et une PR par livrable
- Relue par un autre membre

Le conflit

- Provoqué volontairement
- Résolu à deux

> Votre dépôt est un livrable noté. Il montre comment l'équipe s'est organisée.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com